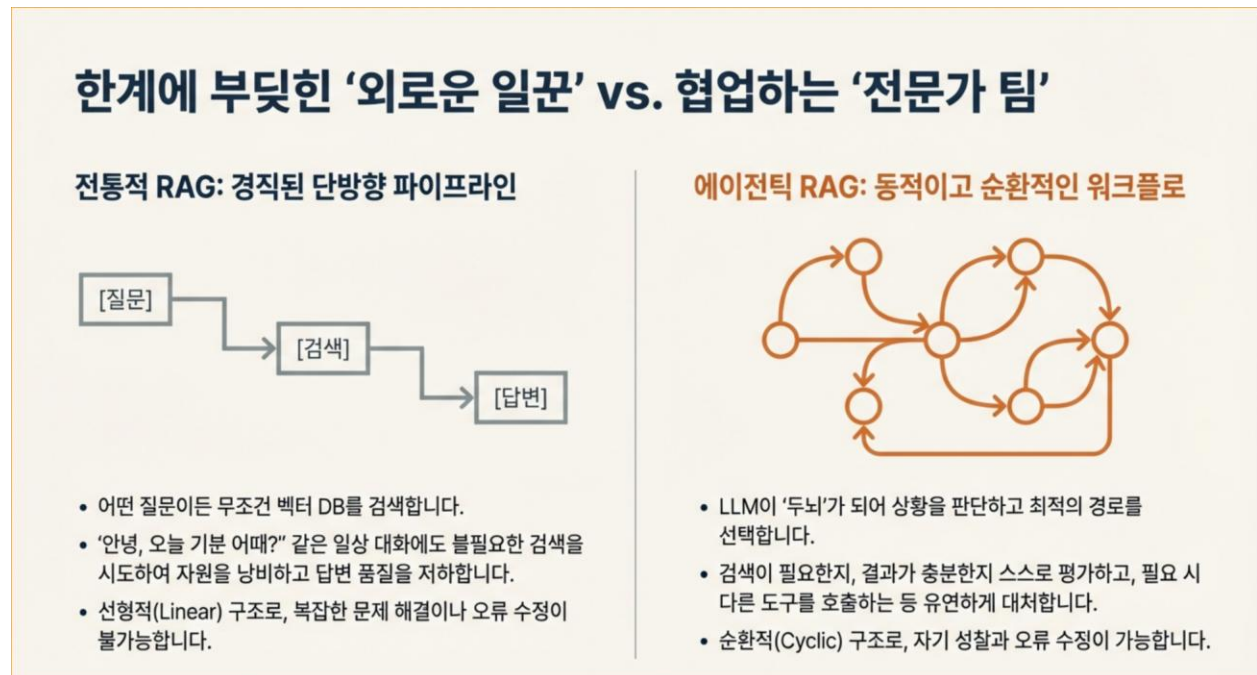


Chapter13

랭그래프(LangGraph)를 활용한 차세대 멀티에이전트 RAG 시스템 구축

1. 서론: 정적 검색에서 동적 에이전트 협업으로의 패러다임 전환



인공지능(AI)과 거대언어모델(LLM) 기술의 급격한 발전 속에서, 검색 증강 생성(RAG, Retrieval-Augmented Generation) 기술은 모델의 환각(Hallucination) 현상을 억제하고 최신 정보를 제공하는 핵심 솔루션으로 자리 잡았습니다. 그러나 초기의 전통적인 RAG 시스템은 사전에 정의된 선형적(Linear) 파이프라인에 의존하여, 사용자의 복잡하고 다층적인 의도를 충분히 반영하지 못한다는 한계에 직면했습니다.

단순한 코드 구현을 넘어, 왜 멀티에이전트 시스템이 필요한지, 그리고 랭그래프(LangGraph)라는 도구가 어떻게 복잡한 비즈니스 로직을 유연한 그래프 구조로 변환시키는지를 탐구할 것입니다.

1.1 전통적 RAG의 구조적 한계와 에이전틱 접근의 필요성

우리가 흔히 접하는 전통적인 RAG 시스템, 즉 'Standard RAG'는 "검색하고 답변한다(Retrieve and Generate)"라는 단순한 명제 아래 작동합니다. 이는 사용자가 어떤 질문을 던지든 시스템이

무조건적으로 벡터 데이터베이스(Vector Store)를 검색하고, 그 결과를 LLM 에 주입하여 답변을 생성하는 '단방향(Single-pass)' 흐름을 의미합니다. 예를 들어, 사용자가 "안녕, 오늘 기분 어때?"라는 일상적인 인사를 건넬 때조차, 전통적인 RAG 는 사내 문서나 데이터베이스에서 '안녕'과 관련된 키워드를 검색하려 시도합니다. 이는 불필요한 컴퓨팅 자원의 낭비일 뿐만 아니라, 문맥에 맞지 않는 엉뚱한 문서를 참고하여 답변의 품질을 떨어뜨리는 원인이 됩니다.

이러한 경직성(Rigidity)은 복잡한 문제 해결 상황에서 더욱 두드러집니다. 사용자의 질문이 모호하거나, 데이터베이스에 정답이 없거나, 여러 단계의 추론이 필요한 경우 전통적 RAG 는 유연하게 대처하지 못하고 낮은 품질의 답변을 내놓거나 "정보가 없습니다"라는 단답형 반응에 그칩니다. 반면, 에이전틱 RAG(Agentic RAG)는 이러한 선형적 구조를 탈피하여 순환적(Cyclic)이고 동적(Dynamic)인 워크플로를 채택합니다. 에이전틱 시스템은 LLM 을 단순한 문장 생성기가 아닌, 추론하고 판단하는 '두뇌'로 활용합니다. 시스템은 사용자의 질문을 분석하여 검색이 필요한지 판단하고(Routing), 검색된 정보가 충분한지 스스로 평가하며(Self-Reflection), 부족하다면 검색어를 수정하여 재검색을 수행하거나 웹 검색과 같은 다른 도구를 호출합니다.

비교 항목	전통적 RAG (Standard RAG)	에이전틱 RAG (Agentic RAG)	기술적 함의
워크플로 구조	선형적, 단방향 (Linear DAG)	순환적, 다중 경로 (Cyclic Graph)	루프(Loop)를 통한 오류 수정 및 품질 개선 가능
의사결정 주체	하드코딩된 규칙 (Developer)	LLM 에이전트 (Autonomous Agent)	런타임 상황에 따른 동적 경로 변경 (Dynamic Routing)
유연성	낮음 (모든 입력에 동일 절차)	높음 (입력 의도에 따른 차별화)	비효율적 리소스 사용 방지 및 사용자 경험 최적화
데이터 소스	단일 데이터 소스 고정	다중 소스 및 도구 활용 (Multi-source)	API, 웹 검색, 내부 DB 등을 상황에 맞춰 선택
확장성	기능 추가 시 파이프라인 재설계 필요	노드(Node) 추가를 통한 모듈식 확장	유지보수 용이성 및 시스템 고도화 유리

위 표에서 볼 수 있듯이, 에이전틱 RAG 는 시스템의 자율성(Autonomy)과 적응성(Adaptability) 측면에서 압도적인 우위를 점합니다. 문서에서 저자는 맥도날드의 '빅맥' 제조 공정을 예로 들어 설명합니다. 복잡한 작업을 하나의 거대 모델에 뭉뚱그려 맡기기보다는, '빵을 굽는 에이전트', '패티를 굽는 에이전트', '조립하는 에이전트'와 같이 업무를 세분화하고 각 단계에 명확한 가이드라인(Prompt)을 제공할 때 비로소 일관된 품질을 담보할 수 있다는 것입니다. 이것이 바로 우리가 랭그래프를 통해 구현하고자 하는 멀티에이전트 시스템의 핵심 철학입니다.

1.2 랭그래프(LangGraph): 에이전트 오케스트레이션의 도구

이러한 에이전틱 워크플로를 구현하기 위해 선택된 도구는 랭그래프(LangGraph)입니다. 랭체인(LangChain)이 일련의 선형적인 호출(Call)을 연결하는 데 최적화되어 있다면, 랭그래프는 상태(State)를 공유하며 여러 에이전트가 협업하고 순환하는 그래프 구조를 설계하는 데 특화되어 있습니다. 랭그래프는 구글의 Pregel 시스템에서 영감을 받아 '슈퍼 스텝(Super-step)' 단위로 노드를 실행하며, 이를 통해 복잡한 멀티 턴(Multi-turn) 대화나 장기 기억(Long-term Memory)이 필요한 작업을 효과적으로 처리합니다. 앞 장의 예제를 기반으로, 라우터(Router), 검색기(Retriever), 평가자(Grader), 생성기(Generator)라는 네 가지 핵심 역할을 정의하고 이를 유기적으로 연결하는 과정을 상세히 다룰 것입니다.

2. 사전 준비: 데이터 파이프라인 구축 및 인프라 설정

모든 것은 데이터에서 시작됩니다: RAG 시스템의 견고한 기반 구축

성공적인 RAG 시스템의 근간은 고품질의 데이터입니다. (Garbage In, Garbage Out)



문서 로딩 및 텍스트 분할 (Document Loading & Splitting)

Tool: PyPDFLoader (서울시/뉴욕시 도시 계획 문서 예제)

Strategy: RecursiveCharacterTextSplitter 사용

Key Parameters & Rationale:

- **Chunk Size: 1900:** 일반적인 RAG(500~1000)보다 큰 크기. 도시 계획 문서처럼 문맥의 호흡이 긴 전문 문서에 더 많은 컨텍스트를 제공하여 답변 품질을 높입니다.
- **Chunk Overlap: 100:** 청크 간 중복 구간을 두어 문장이 잘려 의미가 손실되는 것을 방지합니다.



벡터 DB 구축 및 임베딩 (Vector Store & Embedding)

Tool: Chroma DB & OpenAI text-embedding-3-large

- **배치(Batch) 처리:** 100개 단위로 문서를 처리하여 OpenAI API의 RateLimitError를 방지하고 안정성을 확보합니다.
- **영구 저장소(Persistence):** persist_directory를 설정하여 임베딩된 벡터를 디스크에 저장, 비용과 시간을 절약합니다.

성공적인 RAG 시스템의 근간은 고품질의 데이터입니다. 아무리 뛰어난 에이전트 아키텍처를 설계하더라도, 참조하는 데이터가 엉망이라면 결과물 또한 신뢰할 수 없습니다(Garbage In, Garbage Out). 따라서 우리는 랭그래프를 본격적으로 다루기에 앞서 문서에서 제시하는 PDF 전처리 및 벡터 데이터베이스 구축 과정을 기술적으로 정밀하게 분석하고 수행해야 합니다.

2.1 문서 로딩 및 텍스트 분할 전략 (Document Loading & Splitting)

문서¹에서는 서울시와 뉴욕시의 도시 계획 문서를 예제로 사용합니다. 이 문서들은 복잡한 표와 서식을 포함하고 있을 가능성이 높은 PDF 형식이므로, 이를 텍스트로 추출하고 LLM이 처리하기 적절한 단위로 나누는 과정이 필수적입니다.

2.1.1 PDF 로더의 선택과 활용

강의 예제에서는 PyPDFLoader를 사용합니다.¹ 이는 파이썬 생태계에서 가장 널리 쓰이는 PDF 파싱 도구 중 하나로, 각 페이지를 하나의 Document 객체로 변환합니다. pypdf 라이브러리에 기반한 이 로더는 텍스트 추출 속도가 빠르지만, 복잡한 레이아웃이나 이미지 속 텍스트(OCR이 필요한 경우)를 처리하는 데에는 한계가 있을 수 있습니다. 실무에서는 문서의 특성에 따라 UnstructuredPDFLoader나 Azure Document Intelligence와 같은 고급 도구를 고려해야 할 수도 있습니다.

```
# 기반 코드 재구성
from langchain_community.document_loaders import PyPDFLoader

def load_pdf(pdf_path):
    loader = PyPDFLoader(pdf_path)
    documents = loader.load()
    print(f"Loaded {len(documents)} pages from {pdf_path}")
    return documents
```

2.1.2 재귀적 텍스트 분할 (Recursive Character Text Splitting)

텍스트 분할(Chunking)은 RAG 의 검색 정확도를 좌우하는 매우 중요한 하이퍼파라미터입니다.

예제에서는 RecursiveCharacterTextSplitter 를 사용하며, chunk_size 를 1900, chunk_overlap 을 100 으로 설정했습니다.

- **Chunk Size 1900:** 이는 일반적인 RAG 시스템(보통 500~1000 토큰)에 비해 상당히 큰 크기입니다. 큰 청크는 LLM 에게 더 많은 문맥 정보를 제공할 수 있어 복잡한 질의에 대한 답변 품질을 높일 수 있지만, 검색 시 벡터의 의미가 희석되거나 불필요한 정보가 포함될 위험이 있습니다. 도시 계획 문서와 같이 문맥의 호흡이 긴 전문 문서의 경우 이러한 큰 청크가 유리할 수 있습니다.
- **Chunk Overlap 100:** 청크 간의 중복 구간을 두어 문장이 중간에 잘려 의미가 손실되는 것을 방지합니다. 재귀적 분할기는 줄바꿈($\n$, $\n$), 공백(), 빈 문자열("") 순으로 구분자를 시도하며 텍스트를 의미 단위로 보존하려 노력합니다.

```
# 텍스트 분할 코드 분석
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1900,
    chunk_overlap=100
)
splits = text_splitter.split_documents(data_from_pdf)
```

2.2 벡터 데이터베이스 구축 및 임베딩

분할된 텍스트 청크는 기계가 이해할 수 있는 벡터(숫자 배열)로 변환되어야 합니다. 이 과정을 임베딩(Embedding)이라고 하며, 본 강의에서는 OpenAI 의 text-embedding-3-large 모델과 Chroma 벡터 데이터베이스를 사용합니다.

2.2.1 임베딩 모델의 선정

text-embedding-3-large 는 OpenAI 의 최신 임베딩 모델로, 기존 모델 대비 더 높은 차원을 지원하며 다국어 처리 능력과 의미론적 검색 성능이 뛰어납니다. 벡터의 차원이 높을수록 텍스트의 미세한 뉘앙스를 더 잘 포착할 수 있으나, 저장 공간과 검색 연산 비용이 증가하는 트레이드오프가 존재합니다.

2.2.2 Chroma DB 와 영구 저장소 (Persistence)

위의 코드에서 주목해야 할 점은 persist_directory 설정입니다. 이는 생성된 벡터 인덱스를 메모리가 아닌 디스크에 저장함으로써, 프로그램을 재실행할 때마다 비용이 발생하는 임베딩 과정을 반복하지 않도록 합니다. 또한, 대량의 문서를 처리할 때 발생할 수 있는 OpenAI API 의 RateLimitError 를 방지하기 위해 100 개 단위의 배치(Batch) 처리를 구현한 점은 실무적으로 매우 중요한 패턴입니다.

```
# 벡터 스토어 생성 및 배치 처리 로직
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embedding = OpenAIEmbeddings(model='text-embedding-3-large')
persist_directory = '../chroma_store'

# 배치 처리를 통해 안정성 확보
for i in range(0, len(chunks), 100):
    batch = chunks[i:i+100]
    if vectorstore is None:
        vectorstore = Chroma.from_documents(
            documents=batch,
            embedding=embedding,
            persist_directory=persist_directory
        )
    else:
        vectorstore.add_documents(documents=batch)
```

이러한 데이터 파이프라인 구축은 에이전트가 활동할 수 있는 지식의 기반을 마련하는 단계이며, 이 단계의 견고함이 전체 시스템의 성능을 결정짓습니다.

3. 아키텍처 설계: 랭그래프(LangGraph)의 핵심 원리

데이터 준비가 완료되었다면, 이제 에이전트들의 뇌와 신경망을 설계할 차례입니다. 랭그래프는 상태(State)를 중심으로 노드(Node)와 엣지(Edge)가 상호작용하는 그래프 이론에 기반을 두고 있습니다. 이는 개발자가 복잡한 비즈니스 로직을 시각화하고 제어하는 데 탁월한 환경을 제공합니다.

에이전트 팀의 플레이북, 랭그래프(LangGraph)

랭그래프는 상태(State)를 중심으로 노드(Node)와 엣지(Edge)가 상호작용하는 그래프 이론에 기반합니다. 이는 에이전트들의 '뇌'와 '신경망'을 설계하는 것과 같습니다.

상태(State) - 시스템의 공유 메모리

모든 노드가 공유하고 수정하는 단 하나의 전역 상태(Global State) 객체.

```
from typing import List, TypedDict

class GraphState(TypedDict):
    question: str      # 사용자의 질문 (입력)
    generation: str    # LLM이 생성한 최종 답변 (출력)
    documents: List[str] # 검색된 문서들의 리스트
```

이 구조는 시스템 내 데이터 흐름을 명확하게 정의하고, 각 노드가 독립적으로 상태를 읽고 업데이트할 수 있게 합니다.

노드(Node)와 엣지(Edge) - 행동과 흐름의 정의



노드(Node)

실제 작업을 수행하는 Python 함수 (e.g., LLM 호출, DB 검색). 에이전트의 '행동'입니다.



엣지(Edge)

노드 간의 연결 통로. 특히, **조건부 엣지(Conditional Edge)**는 상태에 따라 다음 행동을 동적으로 결정하는 '판단'의 핵심입니다.

3.1 상태(State): 시스템의 공유 메모리

랭그래프의 가장 큰 특징은 **전역 상태(Global State)** 관리입니다. 기존의 랭체인이 체인 간에 데이터를 전달하기 위해 복잡한 인자 전달 방식을 사용했다면, 랭그래프는 하나의 정의된 스키마(Schema)를 가진 상태 객체를 모든 노드가 공유하고 수정합니다. GraphState 는 다음과 같습니다.

```
from typing import List, TypedDict

class GraphState(TypedDict):
    question: str      # 사용자의 질문 (입력)
    generation: str    # LLM 이 생성한 최종 답변 (출력)
    documents: List[str] # 검색된 문서들의 리스트 (중간 산출물)
```

이 TypedDict 구조는 시스템 내에서 데이터가 어떻게 흐르는지를 명확하게 정의합니다.

1. 사용자가 질문을 입력하면 question 필드가 채워집니다.
2. 검색 에이전트(retrieve)는 question 을 읽고 검색 결과를 documents 필드에 업데이트합니다.
3. 평가 에이전트(grade_documents)는 documents 를 읽어 검증한 뒤, 필터링된 결과로 다시 documents 를 덮어씁니다.
4. 생성 에이전트(generate)는 최종 documents 를 바탕으로 답변을 생성하여 generation 필드에 저장합니다.

이러한 상태 기반 접근은 각 노드를 순수 함수(Pure Function)에 가깝게 설계할 수 있게 하여 테스트와 디버깅을 용이하게 합니다. 각 노드는 현재 상태를 입력받아 변경된 부분만을 반환(Return)하면, 랭그래프 엔진이 이를 전체 상태에 병합(Merge)하는 방식으로 작동합니다.

3.2 노드(Node)와 엣지(Edge): 행동과 흐름의 정의

그래프의 구성 요소인 노드와 엣지는 에이전트의 '행동'과 '판단'을 대변합니다.

- **노드(Node):** 실제 작업을 수행하는 Python 함수입니다. LLM 호출, 데이터베이스 검색, API 요청 등 모든 실질적인 연산이 여기서 일어납니다.
- **엣지(Edge):** 노드와 노드 사이의 연결 통로입니다.
 - **일반 엣지(Normal Edge):** A 작업 후 무조건 B 작업으로 이동하는 확정적 흐름입니다.
 - **조건부 엣지(Conditional Edge):** 현재 상태나 이전 작업의 결과에 따라 다음에 실행할 노드를 동적으로 결정합니다.

조건부 엣지는 에이전틱 RAG 의 지능을 구현하는 핵심 메커니즘입니다. "질문이 일상 대화인가, 전문 지식인가?"라는 판단 결과에 따라 그래프의 흐름이 retrieve 노드로 갈지, casual_talk 노드로 갈지를 결정하는 분기점(Branching)이 바로 이 조건부 엣지를 통해 구현됩니다.

4. 컴포넌트 상세 구현: 에이전트 팀 구성



팀 소개 1: 지능형 관제탑, 라우터(Router)

Role

사용자의 질문을 가장 먼저 분석하여 적절한 전문가에게 작업을 분배합니다.
모든 질문에 대해 불필요한 검색을 수행하는 비효율을 막습니다.

Key Skill

구조화된 출력(Structured Output)을 통한 명확한 의사결정

Problem: LLM에게 자연어로 분류를 요청하면, '제 생각에는...'과 같이 형식이 깨진 응답이 나올 수 있습니다.

Solution: Pydantic 모델을 사용하여 LLM의 출력을 `vectorstore` 또는 `casual\_talk` 중 하나로 엄격하게 강제합니다.

```
from pydantic import BaseModel, Field
from typing import Literal
```

```
class RouteQuery(BaseModel):
    """사용자 쿼리를 가장 관련성 높은 데이터 소스로 라우팅합니다."""
    datasource: Literal["vectorstore", "casual_talk"]
```

'Literal' 타입은 LLM이 정의된 두 가지 선택지 중 하나만 반환하도록 보장합니다.

이제 설계를 바탕으로 각 역할을 수행하는 전문 에이전트(노드)들을 구현하고 분석해 보겠습니다.
이들은 하나의 팀처럼 협업하여 사용자의 질문을 해결합니다.

4.1 라우터(Router): 지능형 관제탑

라우터는 사용자의 질문을 가장 먼저 접수하여 그 성격을 분석하는 관제탑 역할을 합니다. 모든 질문에 대해 검색을 수행하는 비효율을 막기 위해, 라우터는 질문을 '벡터 스토어 검색 필요(vectorstore)'와 '일상 대화(casual_talk)'로 분류합니다.

4.1.1 구조화된 출력(Structured Output)의 활용

LLM에게 자연어로 분류를 요청하면, "제 생각에는 이것은 검색이 필요합니다"와 같이 불필요한 서술어가 포함되거나 형식이 깨질 위험이 있습니다. 이를 방지하기 위해 Pydantic 모델을 사용하여 출력을 엄격하게 제어합니다.

라우팅을 위한 데이터 모델

```
class RouteQuery(BaseModel):
```

```
    """사용자 쿼리를 가장 관련성이 높은 데이터 소스로 라우팅합니다."""
```

```
    datasource: Literal["vectorstore", "casual_talk"] = Field(
```

```
    ...,
```

```
description="사용자 질문에 따라 casual_talk 또는 vectorstore 로 라우팅합니다."  
)
```

Literal 타입을 사용함으로써 LLM 은 오직 정의된 두 가지 문자열 중 하나만을 반환하도록 강제됩니다. `structured_llm_router = model.with_structured_output(RouteQuery)` 코드는 LLM(GPT-4o-mini)이 이 스키마를 따르도록 설정합니다.

4.1.2 라우팅 프롬프트 전략

시스템 프롬프트는 라우터의 판단 기준을 제시합니다.

- "서울, 뉴욕의 발전계획과 관련된 문서는 vectorstore 를 사용하라."
- "일상 대화는 casual_talk 를 사용하라."

이러한 명시적인 지시(Instruction)는 라우터가 모호한 질문(예: "서울의 날씨는?")을 처리할 때도 기준점을 제공합니다.

4.2 검색기(Retriever) 노드



팀 소개 2: 지식의 보고, 검색기(Retriever)

Role

라우터가 'vectorstore' 경로를 지시했을 때 실행됩니다. 벡터 데이터베이스에서 사용자의 질문과 가장 관련성이 높은 문서를 찾아옵니다.

Key Action

1. `GraphState`에서 `question`을 입력받습니다.
2. Chroma DB의 `retriever.invoke(question)`을 호출하여 관련 문서를 검색합니다.
3. 검색된 문서 목록을 `GraphState`의 `documents` 필드에 업데이트하여 다음 에이전트에게 전달합니다.

```
1 def retrieve(state):  
2     print('---RETRIEVE---')  
3     question = state['question']  
4     documents = retriever.invoke(question)  
5     return {"documents": documents, "question": question}
```

검색기 노드는 라우터가 vectorstore 경로를 선택했을 때 실행됩니다. 이 노드는 단순히 벡터 DB의 검색 인터페이스(retriever.invoke)를 호출하여 관련 문서를 가져오고, 이를 상태의 documents 키에 업데이트합니다.

```
def retrieve(state):
    print('---RETRIEVE---')
    question = state['question']
    documents = retriever.invoke(question)
    return {"documents": documents, "question": question}
```

여기서 반환된 딕셔너리는 랭그래프에 의해 기존 상태에 병합됩니다.

4.3 문서 평가자(Grader): 품질 관리자



팀 소개 3: 품질 관리자, 문서 평가자(Grader)

Role
 검색된 문서가 항상 정답을 포함하는 것은 아닙니다. 평가자는 관련 없는 문서(False Positive)를 걸러내어 LLM이 잘못된 정보를 바탕으로 답변하는 것을 방지합니다.

Key Skill
이진 평가(Binary Scoring) 을 통한 노이즈 제거
 Pydantic 모델을 사용하여 각 문서의 관련성을 'yes' 또는 'no'로만 판단하도록 LLM을 제어합니다.

- **프롬프트 전략**: '엄격한 테스트가 아니다. 목표는 잘못된 검색 결과를 걸러내는 것이다.' 라는 지침을 통해, LLM이 너무 까다롭지 않으면서도 명백한 오류는 필터링 하도록 유도합니다.

Impact
 RAG 시스템의 신뢰도(Reliability)를 크게 향상시키는 핵심 단계입니다. 5개의 문서 중 관련 있는 3개만 3개만 남겨 생성 모델에게 더 깨끗한 컨텍스트를 제공합니다.



검색된 문서가 항상 정답을 포함하고 있다는 보장은 없습니다. 키워드 매칭이나 벡터 유사도에 의해 관련 없는 문서가 딸려올 수 있습니다(False Positive). 문서 평가자는 이러한 노이즈를 제거하여 LLM 이 잘못된 정보를 바탕으로 답변하는 것을 방지합니다.

4.3.1 이진 평가(Binary Scoring) 시스템

평가자 역시 GradeDocuments 라는 Pydantic 모델을 사용하여 문서의 관련성 여부를 yes 또는 no 로만 판단합니다.

- **프롬프트 전략**: "엄격한 테스트가 아니다. 목표는 잘못된 검색 결과를 걸러내는 것이다."라는 지침을 주어, LLM 이 너무 까다롭게 문서를 버리는 것을 방지하면서도 명백한 오류를 필터링하도록 유도합니다.

문서 평가 로직 (필터링)

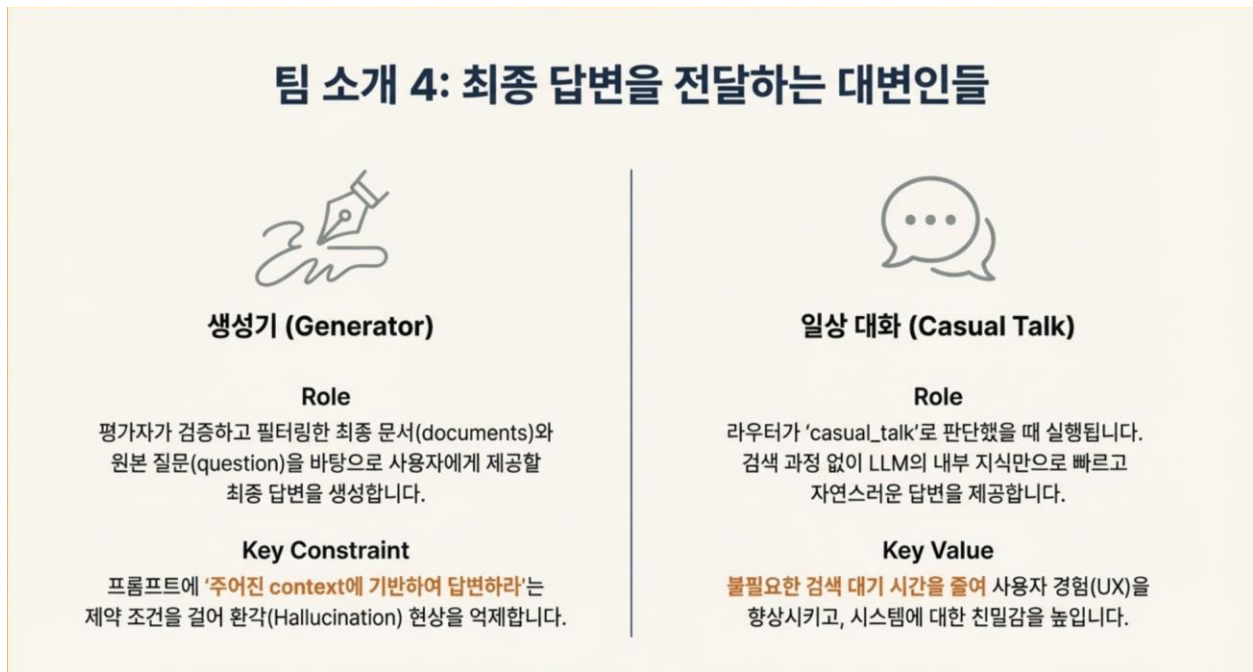
```
def grade_documents(state):
    print('---CHECK RELEVANCE---')
    documents = state['documents']
    question = state['question']
    filtered_docs =

    for doc in documents:
        score = retrieval_grader.invoke({"question": question, "document": doc.page_content})
        if score.binary_score == "yes":
            filtered_docs.append(doc) # 관련 있는 문서만 유지

    return {"documents": filtered_docs, "question": question}
```

이 과정은 RAG 시스템의 신뢰도(Reliability)를 크게 높여주는 핵심 단계입니다. 만약 5 개의 문서를 검색했는데 3 개만 남았다면, 생성 모델은 더 깨끗한 문맥(Context)을 바탕으로 답변을 작성할 수 있게 됩니다.

4.4 생성기(Generator) 및 일상 대화(Casual Talk)

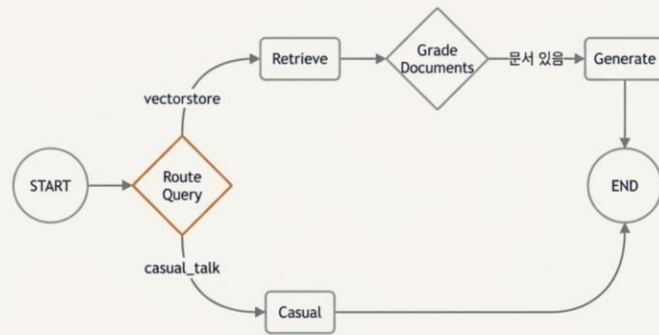


- **Generate 노드:** 필터링된 문서(documents)와 질문(question)을 프롬프트에 넣어 최종 답변을 생성합니다. 이때 프롬프트는 "주어진 context 에 기반하여 답변하라"는 제약 조건을 걸어 할루시네이션을 억제합니다.
- **Casual Talk 노드:** 검색 과정 없이 LLM 의 내부 지식만을 사용하여 답변합니다. 이는 사용자 경험(UX) 측면에서 매우 중요합니다. 사용자가 RAG 시스템에 대해 친밀감을 느끼게 하거나, 간단한 상호작용을 할 때 불필요한 대기 시간을 줄여줍니다.

5. 그래프 통합 및 워크플로 제어

에이전트 팀 조립: 그래프 통합 및 워크플로 제어

1. **노드 등록:** `workflow.add_node()`를 통해 각 에이전트 함수(`retrieve`, `grade_documents` 등)를 그래프에 등록합니다.
2. **진입점 및 조건부 엣지:** `workflow.add_conditional_edges(START, route_question, ...)` 코드가 핵심입니다. `route_question` 함수의 반환값('vectorstore' 또는 'casual_talk')에 따라 다음 경로가 동적으로 결정됩니다.
3. **일반 엣지 연결:** `workflow.add_edge()`를 통해 `retrieve -> grade_documents -> generate`와 같은 확정적 흐름을 연결합니다.
4. **컴파일:** `app = workflow.compile()`을 통해 실행 가능한 애플리케이션을 완성합니다.



개별 노드의 구현이 끝났다면, 이제 이들을 StateGraph 위에서 조립하고 연결할 차례입니다. 이 단계에서 우리는 정적인 코드를 살아있는 유기체와 같은 동적 시스템으로 변환합니다.

5.1 그래프 빌드 및 엣지 연결

StateGraph 구성

```
from langgraph.graph import START, StateGraph, END
```

```
workflow = StateGraph(GraphState)
```

1. 노드 등록

```
workflow.add_node("retrieve", retrieve)
workflow.add_node("grade_documents", grade_documents)
workflow.add_node("generate", generate)
workflow.add_node("casual_talk", casual_talk)
```

2. 엔트리 포인트 및 조건부 엣지 설정

```
workflow.add_conditional_edges(
    START,
    route_question, # 라우팅 함수 실행
    {
        "vectorstore": "retrieve", # 결과가 vectorstore 면 retrieve 노드로
        "casual_talk": "casual_talk" # 결과가 casual_talk 면 casual_talk 노드로
    }
)
```

3. 일반 엣지 연결 (파이프라인 흐름)

```
workflow.add_edge("retrieve", "grade_documents")
workflow.add_edge("grade_documents", "generate")
workflow.add_edge("generate", END)
workflow.add_edge("casual_talk", END)
```

4. 컴파일

```
app = workflow.compile()
```

5.2 조건부 엣지(Conditional Edge)의 작동 원리

위 코드에서 `add_conditional_edges` 는 랭그래프의 강력함을 보여주는 부분입니다. START 노드에서 시작하자마자 `route_question` 함수가 호출됩니다. 이 함수의 반환값(문자열)은 딕셔너리의 키(`vectorstore`, `casual_talk`)와 매핑되어 다음 목적지를 결정합니다. 이는 프로그래밍의 if-else 구문과 유사하지만, 그래프 구조 내에서 시각화되고 추적 가능한 형태로 제어 흐름을 관리한다는 점에서 차이가 있습니다.

5.3 실행 시나리오 분석

시나리오 분석: 팀은 어떻게 상황에 맞춰 협업하는가

Scenario A: 전문적 질문

"서울시 자율주행 계획에 대해 알려줘."



Router's Decision: 질문의 키워드('서울시', '계획')를 분석하여 'vectorstore'를 반환합니다.

Workflow

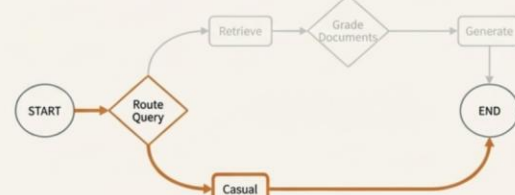
1. Retrieve: 관련 도시 계획 문서를 검색합니다.
2. Grade: 검색된 문서의 관련성을 검증합니다.
3. Generate: 검증된 문서를 기반으로 구체적이고 전문적인 답변을 생성합니다.

Outcome

신뢰할 수 있는 문서에 기반한 고품질 답변을 제공합니다.

Scenario B: 일상적 질문

"안녕, 잘 지내고 있어?"



Router's Decision: 질문의 의도가 인사임을 파악하여 'casual_talk'을 반환합니다.

Workflow

검색 및 평가 과정을 모두 건너뛰고, casual_talk 노드가 즉시 응답합니다.

Outcome

빠른 응답 속도와 자연스러운 대화 흐름으로 뛰어난 사용자 경험을 제공합니다.

1. 시나리오 A: 전문적 질문 ("서울시 자율주행 계획")

- **Route:** 질문 내 키워드('서울시', '계획')를 분석하여 vectorstore 반환.
- **Flow:** retrieve 노드로 이동하여 문서를 검색하고, grade_documents 에서 관련성을 검증한 후, generate 에서 답변을 생성하고 종료합니다.
- **결과:** 사용자는 신뢰할 수 있는 문서에 기반한 구체적이고 전문적인 답변을 얻습니다.¹

2. 시나리오 B: 일상적 질문 ("잘 지내고 있어?")

- **Route:** 질문의 의도가 인사인 것을 파악하여 casual_talk 반환.
- **Flow:** 검색 과정을 건너뛰고 바로 casual_talk 노드로 이동하여 즉시 응답하고 종료합니다.
- **결과:** 빠른 응답 속도와 자연스러운 대화 흐름을 제공합니다.

이러한 분기 처리는 시스템이 '상황 인식(Context-Aware)' 능력을 갖추게 됨을 의미합니다. 사용자는 하나의 인터페이스를 통해 대화와 정보 검색을 동시에 수행할 수 있으며, 시스템은 내부적으로 최적의 경로를 선택하여 효율성을 극대화합니다.

6. 심화: 적응형(Adaptive) RAG 로의 진화와 확장 전략

팀의 진화: 적응형(Adaptive) RAG로의 확장

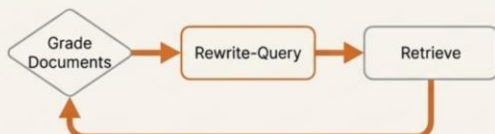
현재 시스템은 훌륭한 기초입니다. 하지만 실제 환경에서는 예측 불가능한 상황이 발생합니다. 랭그래프의 순환(Cycle) 기능을 활용해 시스템을 더욱 똑똑하게 만들 수 있습니다.

자기 성찰(Self-Correction) 루프 - "실수로부터 배우기"

Problem: 'grade_documents' 단계에서 모든 문서가 '관련 없음'으로 필터링되면 어떻게 할까?

Solution:

1. 유효한 문서가 없으면, 'generate'로 가는 대신 '질문 재작성(Rewrite-Query)' 노드를 호출합니다.
2. LLM이 원래 질문을 검색에 더 적합하게 수정합니다. (예: "그거 언제야?" -> "서울시 자율주행 시범 운행 시작일은?")
3. 재작성된 질문으로 다시 'retrieve' 노드를 호출하여 루프(Loop)를 형성합니다.



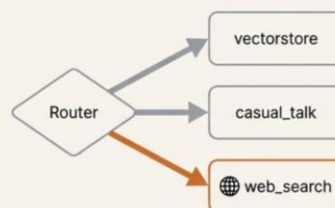
외부 전문가 초빙 - "웹 검색(Web Search) 도구 통합"

Problem:

'오늘 서울 날씨 어때?'와 같이 내부 DB에 없는 최신 정보에 대한 질문.

Solution:

라우터의 선택지에 'web_search'를 추가하고, Tavily나 Google Search API를 호출하는 노드를 연결하여 하이브리드 정보 검색 시스템을 구축합니다.



지금까지 구현한 시스템은 멀티에이전트 RAG의 기초적인 형태입니다. 하지만 실제 프로덕션 환경에서는 더욱 복잡하고 예측 불가능한 상황들이 발생합니다. 예를 들어, 검색된 문서가 모두 관련성이 없다면 어떻게 해야 할까요? 또는 사용자의 질문이 너무 최신 정보라 내부 DB에 없다면요? 여기서 우리는 시스템을 한 단계 더 진화시켜 '적응형 RAG(Adaptive RAG)' 모델로 확장할 수 있습니다.

6.1 자기 성찰(Self-Correction) 루프 구현

앞의 예제에서는 grade_documents 이후 무조건 generate로 넘어갑니다. 하지만 만약 모든 문서가 필터링되어 filtered_docs가 비어있다면, 생성기는 답변할 근거가 없어지게 됩니다. 이때 랭그래프의 '순환(Cycle)' 기능을 활용할 수 있습니다.

● 확장 로직:

1. grade_documents 노드에서 유효한 문서가 하나도 없는지 확인합니다.
2. 없다면, generate로 가는 대신 '질문 재작성(Rewrite-Query)' 노드로 보냅니다.
3. 이 노드는 LLM을 사용하여 원래 질문을 검색에 더 적합한 형태로 변환합니다 (예: "그거 언제야?" -> "서울시 자율주행 시범 운행 시작일은?").
4. 재작성된 질문으로 다시 retrieve 노드를 호출합니다 (Loop).

이러한 피드백 루프는 시스템이 포기하지 않고 스스로 문제를 해결하려는 시도를 하게 만듭니다. 단, 무한 루프를 방지하기 위해 상태(State)에 `retry_count` 등을 추가하여 최대 시도 횟수를 제한하는 안전장치가 반드시 필요합니다.

6.2 웹 검색(Web Search) 도구의 통합

내부 문서(Vector Store)에 없는 정보에 대한 질문이 들어올 경우를 대비해 외부 지식을 활용할 수 있는 경로를 추가해야 합니다.

- **구현:** 라우터의 선택지에 `web_search` 를 추가하고, Tavily 나 Google Search API 를 활용하는 노드를 연결합니다.
- **효과:** "오늘 서울 날씨 어때?"와 같은 질문이나 최신 뉴스에 대한 질문에도 유연하게 대처할 수 있는 하이브리드 정보 검색 시스템이 완성됩니다.

6.3 할루시네이션 평가자 (Hallucination Grader)

답변이 생성된 후(generate), 이 답변이 정말로 검색된 문서를 기반으로 작성되었는지(Groundedness), 그리고 사용자의 질문에 대한 답이 되었는지(Answer Relevance)를 검증하는 최종 관문을 둘 수 있습니다.

- **작동:** LLM 에게 "문서 A, B, C 를 바탕으로 생성된 답변 D 가 사실에 부합하는가?"라고 묻습니다.
- **대응:** 만약 'No'라면, 답변을 재생성하게 하거나 "문서에서 정보를 찾을 수 없습니다"라고 솔직하게 답변하도록 유도합니다. 이는 금융이나 법률 분야와 같이 정확성이 생명인 도메인에서 필수적인 기능입니다.

7. 결론 및 실무 적용 가이드

단순한 RAG 시스템이 랭그래프를 만나 어떻게 지능적이고 적응력 있는 멀티에이전트 시스템으로 진화하는지를 심층적으로 살펴보았습니다.

7.1 핵심 요약

1. **구조의 변화:** 선형적 체인에서 순환적 그래프로의 전환은 AI 애플리케이션에 '판단'과 '반복'이라는 인간적 사고 프로세스를 불어넣습니다.
2. **역할의 분화:** 라우터, 검색기, 평가자, 생성기로 역할을 세분화함으로써 각 모듈의 성능을 개별적으로 최적화하고 디버깅하기 용이해집니다.
3. **데이터의 중요성:** 아무리 뛰어난 그래프라도 전처리된 데이터와 임베딩 품질이 뒷받침되지 않으면 무용지물입니다.

7.2 프로덕션 환경을 위한 제언

실제 서비스에 이 시스템을 배포하기 위해서는 다음과 같은 사항들을 추가로 고려해야 합니다.

- **관측성(Observability):** LangSmith와 같은 도구를 사용하여 각 노드의 입출력과 지연 시간(Latency), 토큰 사용량을 실시간으로 모니터링해야 합니다.¹⁰ 복잡한 그래프일수록 어디서 병목이 발생하는지 파악하기 어렵기 때문입니다.
- **비동기 처리(Async):** 다수의 사용자가 동시에 접속할 경우, 노드의 실행을 비동기(async def)로 처리하여 시스템의 처리량(Throughput)을 높여야 합니다.
- **비용 관리:** GPT-4와 같은 고성능 모델은 라우터나 평가자와 같은 단순 작업에는 과분할 수 있습니다. 노드별로 더 가볍고 저렴한 모델(예: GPT-4o-mini, local LLM)을 배정하여 성능과 비용의 균형을 맞춰야 합니다.

에이전틱 RAG는 이제 막 개화하기 시작한 기술입니다. 여러분이 자신만의 AI 에이전트 팀을 구축하고, 더 똑똑하고 신뢰할 수 있는 서비스를 만드는 데 견고한 토대가 되기를 바랍니다.